

```
!pip install pypdf sentence-transformers faiss-cpu groq streamlit -q
```

```

===== 382.9/382.9 kB 2.1 MB/s eta 0:00:00
===== 18.8/18.8 MB 47.3 MB/s eta 0:00:00
===== 143.7/143.7 kB 9.5 MB/s eta 0:00:00
===== 10.5/10.5 MB 48.1 MB/s eta 0:00:00
===== 11.4/11.4 MB 50.3 MB/s eta 0:00:00

```

```

import os
from google.colab import files

os.makedirs("data", exist_ok=True)

print("Upload your 7 PDF files (college_rules.pdf, exam_rules.pdf, attendance_policy.pdf,")
print("fees.pdf, scholarship.pdf, hostel_rules.pdf, placement_policy.pdf)")

uploaded = files.upload()

for fname in uploaded.keys():
    os.rename(fname, f"data/{fname}")

print("\nFiles in data/ folder:")
print(os.listdir("data"))

```

Upload your 7 PDF files (college_rules.pdf, exam_rules.pdf, attendance_policy.pdf, fees.pdf, scholarship.pdf, hostel_rules.pdf, placement_policy.pdf)

7 files

college_rules.pdf(application/pdf) - 2733 bytes, last modified: 8/18/2026 - 100% done
attendance_policy.pdf(application/pdf) - 2520 bytes, last modified: 8/18/2026 - 100% done
placement_policy.pdf(application/pdf) - 2514 bytes, last modified: 8/18/2026 - 100% done
hostel_rules.pdf(application/pdf) - 2599 bytes, last modified: 8/18/2026 - 100% done
exam_rules.pdf(application/pdf) - 2727 bytes, last modified: 8/18/2026 - 100% done
scholarship.pdf(application/pdf) - 2613 bytes, last modified: 8/18/2026 - 100% done
fees.pdf(application/pdf) - 2494 bytes, last modified: 8/18/2026 - 100% done
 Saving college_rules.pdf to college_rules.pdf
 Saving attendance_policy.pdf to attendance_policy.pdf
 Saving placement_policy.pdf to placement_policy.pdf
 Saving hostel_rules.pdf to hostel_rules.pdf
 Saving exam_rules.pdf to exam_rules.pdf
 Saving scholarship.pdf to scholarship.pdf
 Saving fees.pdf to fees.pdf

Files in data/ folder:
 ['college_rules.pdf', 'hostel_rules.pdf', 'exam_rules.pdf', 'placement_policy.pdf', 'scholarship.pdf', 'attendance_policy.pdf',

```

from pypdf import PdfReader
import os

def load_pdfs(folder="data"):
    documents = []
    for filename in os.listdir(folder):
        if filename.endswith(".pdf"):
            path = os.path.join(folder, filename)
            reader = PdfReader(path)
            text = ""
            for page in reader.pages:
                text += page.extract_text() + "\n"
            documents.append({"filename": filename, "text": text})
    return documents

documents = load_pdfs("data")

for doc in documents:
    print(f"{doc['filename']} -> {len(doc['text'])} characters extracted")

```

college_rules.pdf -> 1484 characters extracted
 hostel_rules.pdf -> 1253 characters extracted
 exam_rules.pdf -> 1594 characters extracted
 placement_policy.pdf -> 1248 characters extracted
 scholarship.pdf -> 1405 characters extracted
 attendance_policy.pdf -> 1255 characters extracted
 fees.pdf -> 1179 characters extracted

```

def chunk_text(text, chunk_size=300, overlap=50):
    words = text.split()

```

```

chunks = []
start = 0
while start < len(words):
    end = start + chunk_size
    chunk = " ".join(words[start:end])
    chunks.append(chunk)
    start += chunk_size - overlap
return chunks

all_chunks = []      # stores chunk text
chunk_sources = []   # stores which PDF each chunk came from

for doc in documents:
    chunks = chunk_text(doc["text"])
    all_chunks.extend(chunks)
    chunk_sources.extend([doc["filename"]] * len(chunks))

print(f"Total chunks created: {len(all_chunks)}")
print("\nExample chunk:\n", all_chunks[0])

```

Total chunks created: 8

Example chunk:

College General Rules & Regulations 1. Conduct and Discipline 1. Students must behave with courtesy and respect towards faculty

```

from sentence_transformers import SentenceTransformer

embed_model = SentenceTransformer("all-MiniLM-L6-v2")

chunk_embeddings = embed_model.encode(all_chunks, show_progress_bar=True)

print("Embedding shape:", chunk_embeddings.shape)

```

```

modules.json: 100%                               349/349 [00:00<00:00, 21.7kB/s]

config_sentence_transformers.json: 100%           116/116 [00:00<00:00, 13.6kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md: 100%                                   10.5k/10.5k [00:00<00:00, 810kB/s]

sentence_bert_config.json: 100%                   53.0/53.0 [00:00<00:00, 5.37kB/s]

config.json: 100%                                612/612 [00:00<00:00, 34.5kB/s]

model.safetensors: reconstructing file: 100%      90.9MB / 90.9MB, 8.48MB/s

model.safetensors: downloading bytes:            85.0MB, 7.56MB/s

Loading weights: 100%                            103/103 [00:00<00:00, 3516.82it/s]

tokenizer_config.json: 100%                       350/350 [00:00<00:00, 29.0kB/s]

vocab.txt: 100%                                   232k/232k [00:00<00:00, 13.5MB/s]

tokenizer.json: 100%                              466k/466k [00:00<00:00, 22.9MB/s]

special_tokens_map.json: 100%                    112/112 [00:00<00:00, 13.2kB/s]

config.json: 100%                                190/190 [00:00<00:00, 19.9kB/s]

Batches: 100%                                     1/1 [00:01<00:00, 1.40s/it]

Embedding shape: (8, 384)

```

```

import faiss
import numpy as np

dimension = chunk_embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(chunk_embeddings))

print("Total vectors stored in FAISS:", index.ntotal)

```

Total vectors stored in FAISS: 8

```

def retrieve_chunks(query, top_k=3):
    query_embedding = embed_model.encode([query])
    distances, indices = index.search(np.array(query_embedding), top_k)

```

```

results = []
for idx in indices[0]:
    results.append({
        "text": all_chunks[idx],
        "source": chunk_sources[idx]
    })
return results

# Quick test
test_results = retrieve_chunks("How much attendance is required for exams?")
for r in test_results:
    print(f"{r['source']}\n{r['text']}\n")

```

[attendance_policy.pdf]
Attendance Policy 1. Minimum Attendance Requirement 1. Students must maintain at least 75 percent attendance in each subject to

[exam_rules.pdf]
for a supplementary examination as per the academic calendar.

[exam_rules.pdf]
Examination Rules & Guidelines 1. Before the Examination 1. Students must carry their college identity card and admit card to ev

```

import os
from groq import Groq

os.environ["GROQ_API_KEY"] = "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"

client = Groq(api_key=os.environ["GROQ_API_KEY"])

def ask_llm(context, question):
    prompt = f"""You are a helpful college assistant. Answer the question ONLY using the context below.
    If the answer is not in the context, say "I don't have that information in the college documents."

    CONTEXT:
    {context}

    QUESTION:
    {question}

    ANSWER: """

    response = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
    )
    return response.choices[0].message.content

```

```

def rag_chatbot(question, top_k=3):
    retrieved = retrieve_chunks(question, top_k=top_k)
    context = "\n\n".join([r["text"] for r in retrieved])
    answer = ask_llm(context, question)

    sources = list(set([r["source"] for r in retrieved]))
    return answer, sources

# Test it
question = "How much attendance do I need for exams?"
answer, sources = rag_chatbot(question)

print("Q:", question)
print("A:", answer)
print("Sources:", sources)

```

Q: How much attendance do I need for exams?
A: You need to maintain at least **75 percent attendance** in each subject to be eligible to appear for examinations.
Sources: ['attendance_policy.pdf', 'exam_rules.pdf']

```

test_questions = [
    "What is the minimum attendance requirement?",
    "Can I appear for exams with 70% attendance?",
    "What documents are required for medical leave?",
    "What are the examination hall rules?",
    "Can students apply for attendance condonation?",
    "What is the eligibility for placement drives?",

```

```

    "What is the hostel gate closing time?",
]

for q in test_questions:
    answer, sources = rag_chatbot(q)
    print(f"Q: {q}\nA: {answer}\nSources: {sources}\n{'-'*80}")

```

```

Q: What is the minimum attendance requirement?
A: The minimum attendance requirement is 75 percent.
Sources: ['placement_policy.pdf', 'college_rules.pdf', 'attendance_policy.pdf']
-----
Q: Can I appear for exams with 70% attendance?
A: You are below the required 75 % attendance, so you cannot automatically sit for the semester exams. However, because your att
Sources: ['attendance_policy.pdf', 'exam_rules.pdf']
-----
Q: What documents are required for medical leave?
A: A medical certificate from a registered practitioner is required for medical leave.
Sources: ['attendance_policy.pdf', 'exam_rules.pdf']
-----
Q: What are the examination hall rules?
A: **Examination Hall Rules (as per the college documents)**

**Before entering the hall**
- Carry your college identity card and admit card.
- Arrive at least 15 minutes before the scheduled start time.
- If you are more than 30 minutes late, you will not be allowed to enter.
- Students with attendance below the required minimum may be barred from appearing.

**During the examination**
- No mobile phones, smart watches, or any electronic devices are allowed inside the hall.
- Sit only in the seat assigned to you on the seating chart.
- Any form of malpractice (e.g., copying, possessing unauthorized material) will result in cancellation of the exam and discipli
- Do not communicate with other candidates.
- Do rough work only on the sheets provided by the invigilator.

**After the examination**
- Hand over your answer sheet to the invigilator before leaving.
- You may not leave the hall during the first 30 minutes or the last 10 minutes of the exam.
- Requests for re-valuation or re-checking must be submitted within 7 days of result declaration, along with the prescribed fee.
Sources: ['attendance_policy.pdf', 'exam_rules.pdf']
-----
Q: Can students apply for attendance condonation?
A: Yes. Students whose attendance falls between 65 % and 74 % may apply for attendance condonation, following the college's pres
Sources: ['placement_policy.pdf', 'college_rules.pdf', 'attendance_policy.pdf']
-----
Q: What is the eligibility for placement drives?
A: The eligibility for placement drives is:

- A minimum aggregate of 60 percent marks (or equivalent CGPA).
- No active backlogs at the time of registration.
- At least 75 percent attendance.
Sources: ['placement_policy.pdf', 'attendance_policy.pdf', 'exam_rules.pdf']
-----
Q: What is the hostel gate closing time?
A: The hostel gate closes at **9:30 PM on weekdays** and **10:30 PM on weekends**.
Sources: ['hostel_rules.pdf', 'exam_rules.pdf']
-----

```

```

import gradio as gr
import os
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from groq import Groq

# =====
# 1. LOAD GROQ API KEY
# =====

os.environ["GROQ_API_KEY"] = "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"

GROQ_API_KEY = os.environ.get("GROQ_API_KEY")

if not GROQ_API_KEY:
    raise ValueError("GROQ_API_KEY is not set.")

client = Groq(api_key=GROQ_API_KEY)

```

```

# =====
# 2. GROQ MODEL
# =====

MODEL_NAME = "openai/gpt-oss-120b"

# =====
# 3. LOAD PDFs AND BUILD RAG INDEX
# =====

def load_index(folder="data"):

    print("Loading embedding model...")

    embed_model = SentenceTransformer("all-MiniLM-L6-v2")

    all_chunks = []
    chunk_sources = []

    if not os.path.exists(folder):
        raise FileNotFoundError(
            f"Folder '{folder}' not found. "
            "Create a data folder and put your PDFs inside it."
        )

    print("Reading PDFs...")

    for filename in os.listdir(folder):

        if filename.lower().endswith(".pdf"):

            filepath = os.path.join(folder, filename)

            print(f"Reading: {filename}")

            reader = PdfReader(filepath)

            text = ""

            for page in reader.pages:

                page_text = page.extract_text()

                if page_text:
                    text += page_text + "\n"

            words = text.split()

            # Create chunks
            for start in range(0, len(words), 250):

                chunk = " ".join(
                    words[start:start + 300]
                )

                if chunk.strip():

                    all_chunks.append(chunk)
                    chunk_sources.append(filename)

    if not all_chunks:
        raise ValueError(
            "No text was extracted from the PDFs."
        )

    print(
        f"Created {len(all_chunks)} chunks "
        f"from {len(set(chunk_sources))} PDF(s)."
    )

# =====
# CREATE EMBEDDINGS
# =====

```

```

print("Creating embeddings...")

embeddings = embed_model.encode(
    all_chunks,
    show_progress_bar=True
)

# FAISS expects float32
embeddings = np.array(
    embeddings
).astype("float32")

dim = embeddings.shape[1]

index = faiss.IndexFlatL2(dim)

index.add(embeddings)

print("FAISS index created successfully.")

return (
    embed_model,
    index,
    all_chunks,
    chunk_sources
)

# Load everything once
embed_model, index, all_chunks, chunk_sources = load_index()

# =====
# 4. RETRIEVAL FUNCTION
# =====

def retrieve(query, top_k=3):

    # Convert query into embedding
    q_emb = embed_model.encode([query])

    q_emb = np.array(
        q_emb
    ).astype("float32")

    # Search FAISS
    distances, idxs = index.search(
        q_emb,
        top_k
    )

    results = []

    for i in idxs[0]:

        if i >= 0 and i < len(all_chunks):

            results.append(
                (
                    all_chunks[i],
                    chunk_sources[i]
                )
            )

    return results

# =====
# 5. RAG CHAT FUNCTION
# =====

def rag_chat(message, history):

    if not message or not message.strip():

        return "Please enter a question."

```

```

# -----
# RETRIEVE RELEVANT DOCUMENT CHUNKS
# -----

results = retrieve(
    message,
    top_k=3
)

# -----
# BUILD CONTEXT
# -----

context = "\n\n".join(
    [
        result[0]
        for result in results
    ]
)

# -----
# GET SOURCE FILES
# -----

sources = list(
    set(
        [
            result[1]
            for result in results
        ]
    )
)

# -----
# CREATE PROMPT
# -----

prompt = f"""
You are a college assistant chatbot.

Answer ONLY using the information provided
in the CONTEXT below.

If the answer cannot be found in the context,
say exactly:

"I don't have that information in the college documents."

Do NOT make up information.
Do NOT use outside knowledge.

CONTEXT:
{context}

QUESTION:
{message}

ANSWER:
"""

# -----
# SEND TO GROQ
# -----

response = client.chat.completions.create(
    model=MODEL_NAME,
    messages=[
        {
            "role": "user",
            "content": prompt

```

```

    }
    ],

    temperature=0.2
)

# -----
# GET ANSWER
# -----

answer = response.choices[0].message.content

# -----
# ADD SOURCES
# -----

if sources:

    source_text = (
        "\n\n 📖 **Sources:** "
        + ", ".join(sources)
    )

else:

    source_text = ""

full_reply = answer + source_text

return full_reply

# =====
# 6. GRADIO CHAT INTERFACE
# =====

demo = gr.ChatInterface(

    fn=rag_chat,

    title="🎓 College Assistant Chatbot",

    description=(
        "Ask questions about college rules, exams, "
        "attendance, fees, hostel, scholarships, "
        "or placements."
    ),

    examples=[
        "How much attendance do I need for exams?",
        "What is the hostel gate closing time?",
        "What is the eligibility for placement drives?",
        "Can I get a scholarship refund?",
    ],
)

# =====
# 7. LAUNCH
# =====

demo.launch(
    share=True,
    debug=True
)

```


Loading embedding model...

Loading weights: 100%

103/103 [00:00<00:00, 2756.33it/s]

Reading PDFs...

Reading: college_rules.pdf

Reading: hostel_rules.pdf

Reading: exam_rules.pdf

Reading: placement_policy.pdf

Reading: scholarship.pdf

Reading: attendance_policy.pdf

Reading: fees.pdf

Created 8 chunks from 7 PDF(s).

Creating embeddings...

Batches: 100%

1/1 [00:00<00:00, 1.03it/s]

FAISS index created successfully.

Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in la

* Running on public URL: <https://92eaaf6137390682d2.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gra



College Assistant Chatbot

Ask questions about college rules, exams, attendance, fees, hostel, scholarships, or placements.

Chatbot

How much attendance do I need for exams?

What is the hostel gate closing time?

What is the eligibility for placement drives?

Can I get a scholarship refund?

Next steps:



Deploy to Cloud Run